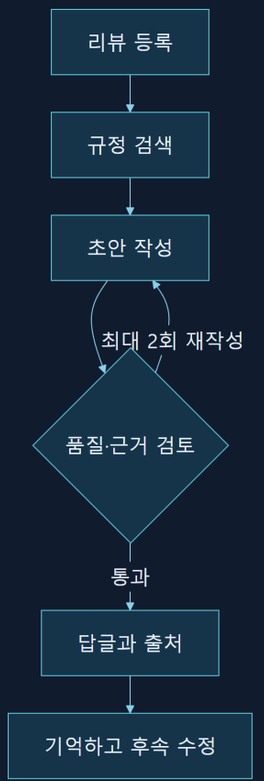
LangChain 수업이 우리 서버가 되는 과정

05·06·07·10·11·12·13 노트북과 branch1·2·3를 연결하는 개발 주석집

FLOW / 흐름을 먼저 읽으세요



이 자료의 질문

임베딩·벡터 DB·그래프·메모리를 왜 쓰는지, 어느 함수에서 실행되는지, 실제 요청이 어떤 순서로 흐르는지 설명합니다.

읽는 순서

그림으로 전체 흐름을 본 뒤 오른쪽의 코드 역할을 읽으세요. 코드 상자는 설명용 발췌·축약 표현이며 실제 실행 코드는 매 페이지 하단의 파일·함수에서 찾습니다.

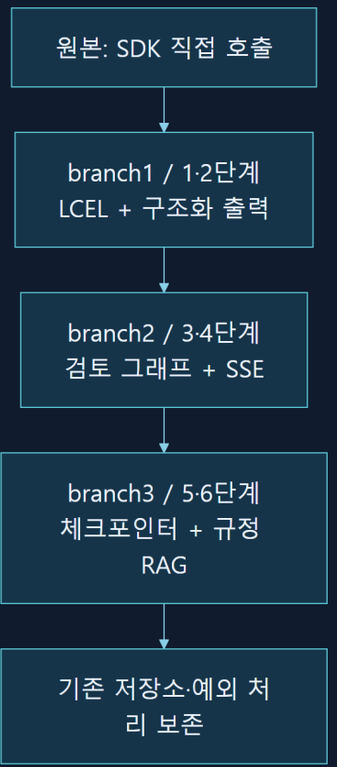
검증 범위

2026-11-13 기준 branch3 테스트 38개와 하위 테스트 7개 통과. 실제 Chroma·로컬 Uvicorn을 사용했습니다. 유료 모델의 의미 품질 평가는 별도입니다.

세 브랜치는 누적된 세 가지 완성본

앞 단계의 HTTP 계약을 보존하며 필요한 기능을 확장했습니다.

FLOW / 흐름을 먼저 읽으세요



branch1: 1·2단계

직접 SDK 호출을 LCEL로 바꾸고 reply·tone·needs_followup을 Pydantic 객체로 받습니다. 문자열 체인은 1단계 커밋 7352eff에서 비교할 수 있습니다.

branch2: 3·4단계

초안과 구조화 검토를 연결하고 재작성 상한을 둡니다. 기존 단건 API 옆에 SSE 스트리밍 API를 추가합니다.

branch3: 5·6단계

리뷰별 이전 초안을 기억하고 짧은 수정 지시를 처리합니다. 매장 규정을 먼저 검색해 작성·검토 프롬프트에 주입합니다.

유지한 경계

원본 저장소와 전역 404·502 처리기는 보존합니다. 기존 등록·조회 라우터는 def, 답글·SSE·수정 라우터는 async def입니다.

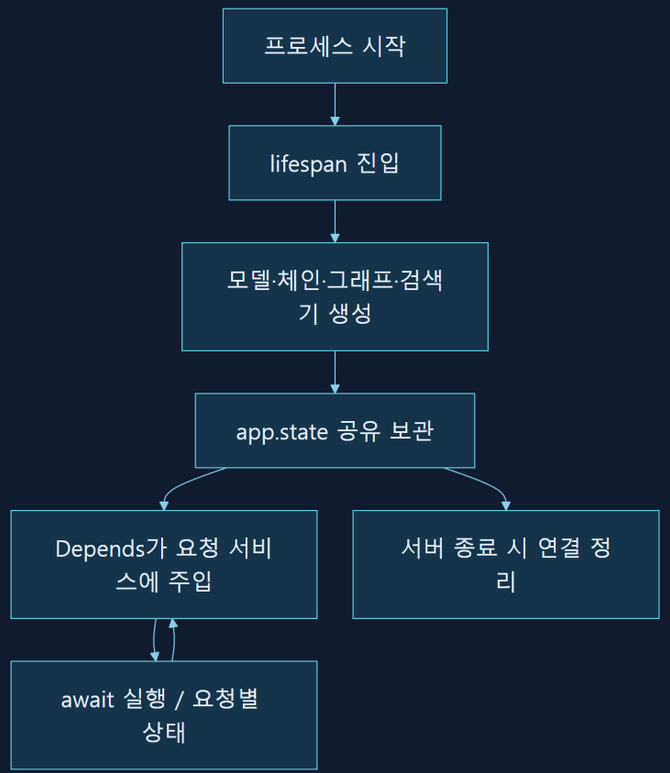
우리 코드 app/routers/review_router.py

수업 원본 complete/10_langgraph_basics.ipynb · fa65250 (클릭)

생성은 서버 시작 때, 실행은 요청마다

공유 객체와 요청별 상태를 구분하면 중복 생성과 동시 요청 혼선을 줄일 수 있습니다.

FLOW / 흐름을 먼저 읽으세요



lifespan은 준비·정리 구간

모델·체인·그래프·체크포인터·인덱스를 한 번 조립합니다. 종료할 때 HTTP와 Chroma 연결을 닫습니다. reload나 여러 worker에서는 프로세스마다 따로 생성됩니다.

app.state와 Depends

app.state는 공유 보관함입니다. 의존성 함수가 같은 객체를 꺼내 가벼운 서비스 객체에 넣습니다. 요청마다 모델을 새로 만들지 않습니다.

공유하면 안 되는 것

초안, 검토 피드백, 시도 횟수는 그래프 실행 상태에 둡니다. 같은 리뷰의 기억 수정은 리뷰별 asyncio.Lock으로 직렬화합니다.

설명용 발췌·축약 코드 / 실제 함수는 페이지 하단 참조

```
application.state.reply_graph = build_graph(
    client, checkpointer=saver,
    policy_retriever=index.retriever(k),
)
```

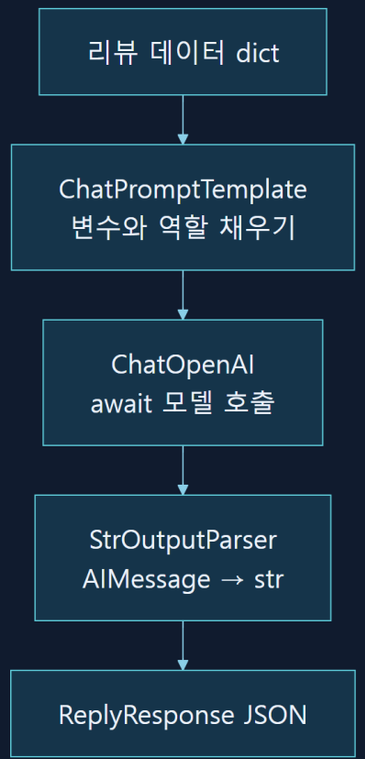
우리 코드 app/main.py:18 / create_app

수업 원본 complete/10_langgraph_basics.ipynb · fa65250 (클릭)

branch1: prompt | llm | parser

수업 01·02의 기본기를 이후 RAG·그래프에서도 그대로 사용합니다.

FLOW / 흐름을 먼저 읽으세요



템플릿: 질문지 만들기

ChatPromptTemplate은 system 역할과 human 데이터의 자리를 정의합니다. 리뷰 JSON은 변수 값으로 전달하므로 중괄호·줄바꿈이 다시 템플릿 명령이 되지 않습니다.

모델: 답변 작성하기

ChatOpenAI가 비동기로 모델을 호출합니다. 라우터에서 기다리는 전체 경로는 await chain.ainvoke(inputs)로 연결됩니다.

파서: 봉투에서 본문 꺼내기

StrOutputParser는 AIMessage에서 문자열을 꺼냅니다. branch2·3의 draft 체인도 이 형태이며 그래프가 흐름을 제어합니다.

설명용 발췌·축약 코드 / 실제 함수는 페이지 하단 참조

```
chain = prompt | client.llm | StrOutputParser()
reply = await chain.ainvoke(inputs)
```

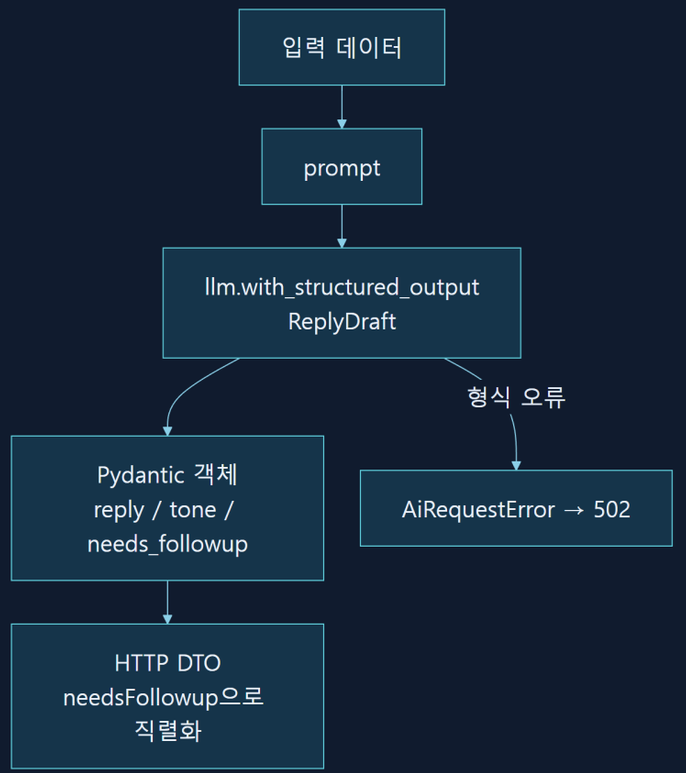
우리 코드 app/workflows/reply_graph.py:58 / build_graph

수업 원본 complete/02_prompt_and_lcel.ipynb · fa65250 (클릭)

구조화 출력: 문자열에 이름표 붙이기

형식 검증과 의미 검토는 서로 다른 역할입니다.

FLOW / 흐름을 먼저 읽으세요



우리 출력 계약

ReplyDraft는 reply 문자열, 사과·감사·일반 중 tone, 후속 확인 여부인 needs_followup을 요구합니다. branch1의 체인은 이 객체를 반환합니다.

API 응답으로 바꾸기

HTTP의 CamelModel 규칙에 따라 needs_followup은 needsFollowup이 됩니다. 모델 출력 스키마와 외부 응답 DTO는 별개입니다.

그래프에서는 검토에도 활용

ReplyGrade는 passed·reason·tone·needs_followup을 검증합니다. Pydantic이 모양을 검증해도 답글이 실제 규정에 맞는지까지 자동으로 증명하지는 않습니다.

설명용 발췌·축약 코드 / 실제 함수는 페이지 하단 참조

```
structured = llm.with_structured_output(
    ReplyDraft, method="json_schema", strict=True
)
chain = prompt | structured
```

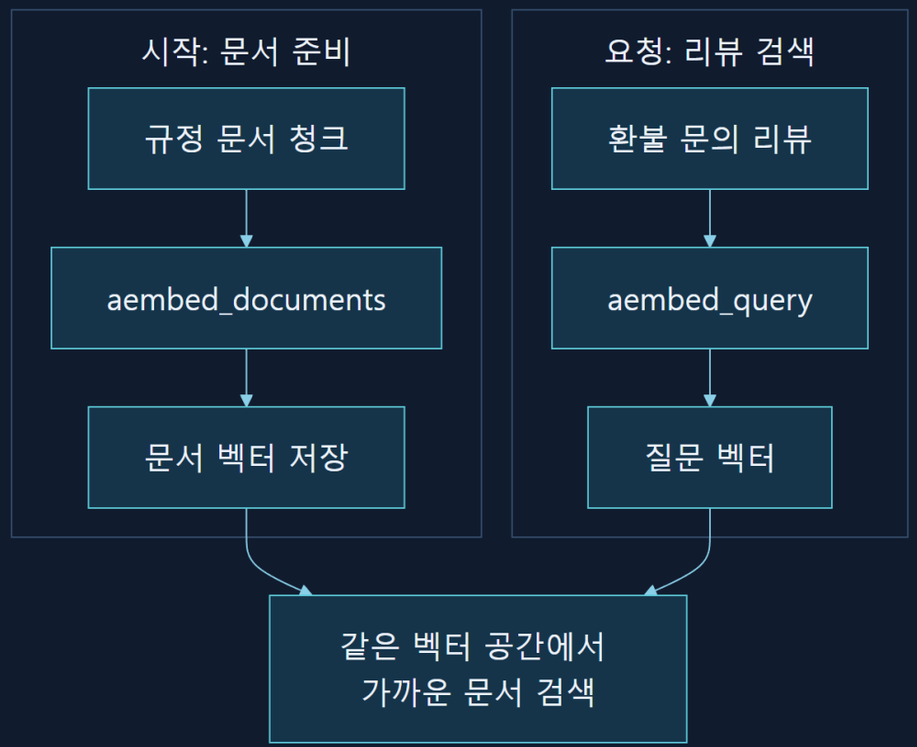
우리 코드 app/services/reply_service.py:23 / build_reply_chain

수업 원본 complete/03_structured_output.ipynb · fa65250 (클릭)

05 임베딩: 문장을 검색용 숫자로 바꾸기

수업 §2 문장 변환과 §4 의미 검색을 서버의 두 실행 시점으로 분리했습니다.

FLOW / 흐름을 먼저 읽으세요



문서 임베딩

규정 청크 여러 개를 aembed_documents로 바꿔 저장합니다. 이 작업은 매 요청이 아니라 문서 인덱스를 처음 준비할 때 수행합니다.

질문 임베딩

리뷰 한 개는 aembed_query로 바꿉니다. 문서와 질문이 같은 모델의 같은 벡터 공간에 있어야 거리 비교가 의미 있습니다.

수업과 우리 설정의 차이

수업 예제는 large 모델을 사용합니다. 우리 기본 설정은 text-embedding-3-small이며 모델명은 설정으로 바꿉니다. 데모 해시 벡터는 실제 의미 임베딩이 아닙니다.

거리는 정답 확률이 아님

수업 05의 코사인은 개념 학습입니다. Chroma의 거리·유사도 해석은 설정에 따라 달라집니다. 가까운 문서라도 답에 필요한 규정인지 별도 확인이 필요합니다.

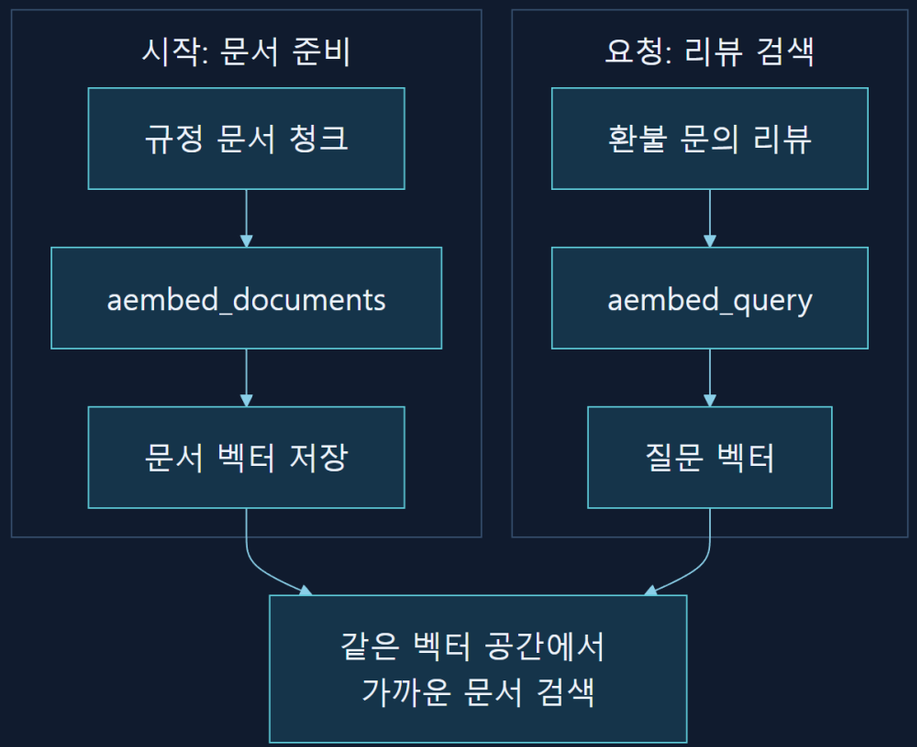
우리 코드 app/rag/policy_index.py:99 / build_policy_index

수업 원본 complete/05_embedding.ipynb · fa65250 (클릭)

05가 반영된 실제 비동기 경로

이벤트 루프에서는 네트워크를 기다리고, 로컬 동기 검색은 작업 스레드로 보냅니다.

FLOW / 흐름을 먼저 읽으세요



문서 준비 코드

build_policy_index가 await embeddings.aembed_documents를 호출합니다. 먼저 얻은 벡터를 Chroma 공개 collection.add에 넣어 동기 원격 호출이 숨어 실행되지 않게 합니다.

요청 검색 코드

PolicyRetriever._aget_relevant_documents에서 질문만 비동기로 임베딩합니다. 이어서 similarity_search_by_vector의 동기 작업을 to_thread.run_sync로 분리합니다.

왜 이렇게 바꿨나요?

노트북은 실행 결과를 보는 단독 프로그램이고 FastAPI는 여러 요청을 함께 처리합니다. async def 안에서 embed_query 같은 동기 HTTP 호출을 직접 실행하면 다른 요청이 막힐 수 있습니다.

설명용 발췌·축약 코드 / 실제 함수는 페이지 하단 참조

```
vectors = await embeddings.aembed_documents(texts)

vector = await self.embeddings.aembed_query(query)
docs = await to_thread.run_sync(
    lambda: self.store.similarity_search_by_vector(
        vector, k=self.k
    )
)
```

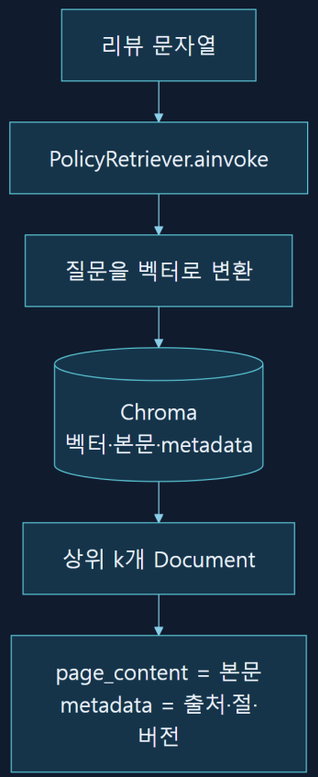
우리 코드 app/rag/policy_index.py:45 / PolicyRetriever

수업 원본 complete/05_embedding.ipynb · fa65250 (클릭)

06 벡터 스토어와 리트리버는 다른 역할

수업 §3 저장, §4 검색, §6 표준 검색기를 우리 PolicyRetriever로 연결했습니다.

FLOW / 흐름을 먼저 읽으세요



Chroma: 책장

벡터와 함께 규정 본문·출처를 저장하고 가까운 벡터를 찾습니다. 벡터만 저장하면 답글에 실제 규정 문장을 넣을 수 없습니다.

Retriever: 책을 찾아주는 창구

입력 문자열을 받아 Document 목록을 돌려주는 인터페이스입니다. 우리 구현은 BaseRetriever의 비동기 메서드를 구현해 ainvoke로 호출합니다.

Document의 두 칸

page_content는 프롬프트에 넣을 본문입니다. metadata는 source·section·chunk_id·policy_version처럼 본문의 위치와 버전을 설명합니다.

k=4의 의미

최대 네 개 후보를 가져오겠다는 뜻입니다. 네 개가 전부 관련 문서라는 뜻은 아닙니다. 이번 구현에는 별도 문서 관련성 채점·MMR·임계값 보정이 없습니다.

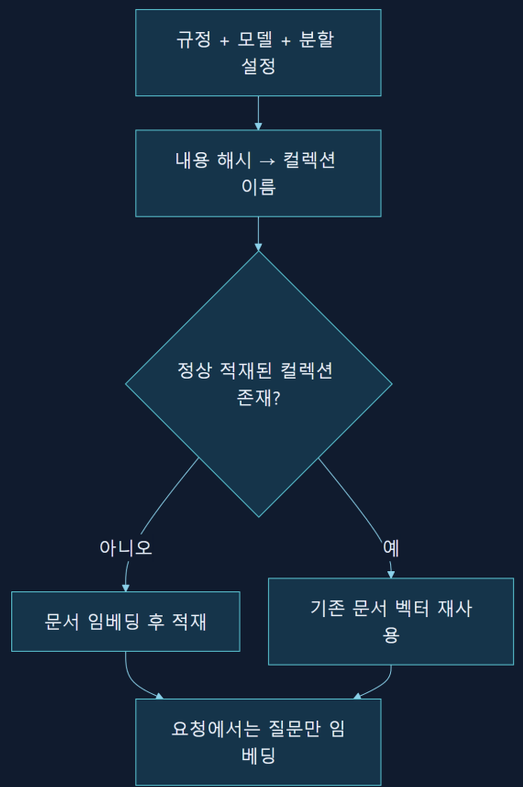
우리 코드 app/rag/policy_index.py:45 / PolicyRetriever

수업 원본 complete/06_vectorstore_and_retriever.ipynb · fa65250 (클릭)

06의 영구 저장을 서버 생명 주기에 넣기

같은 규정의 문서 벡터를 다시 만들지 않도록 했습니다.

FLOW / 흐름을 먼저 읽으세요



실제 모드

PersistentClient가 .chroma에 저장합니다. 규정 내용·파일명·임베딩 모델·분할 버전을 해시한 이름으로 컬렉션을 찾습니다. 적재 건수도 검사합니다.

문서가 바뀌면

해시가 달라져 새 컬렉션을 만듭니다. 다른 벡터 공간을 한 컬렉션에 섞지 않습니다. 기존 컬렉션 자동 삭제·발행 포인터·롤백은 이번 범위가 아닙니다.

테스트의 의미

실제 디스크 Chroma를 닫고 다시 열어, 문서 임베딩 호출 없이 검색이 이어지는지 검사했습니다. 원격 임베딩 HTTP는 대역이므로 실제 모델 검색 품질 측정과 구별합니다.

설명용 발췌·축약 코드 / 실제 함수는 페이지 하단 참조

```
stored = await to_thread.run_sync(collection.count)
if stored not in (0, len(chunks)):
    raise ValueError("incomplete index")
if stored == 0:
    vectors = await embeddings.aembed_documents(texts)
```

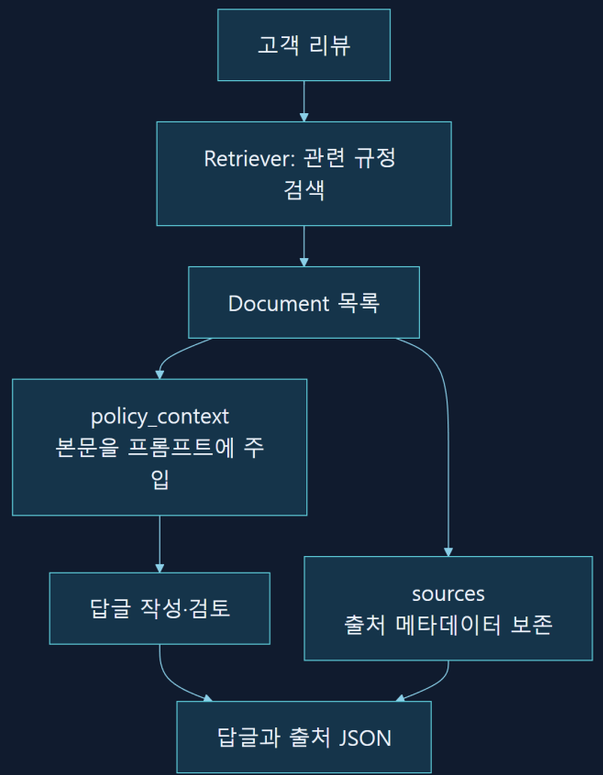
우리 코드 app/rag/policy_index.py:99 / build_policy_index

수업 원본 complete/06_vectorstore_and_retriever.ipynb · fa65250 (클릭)

07 RAG: 찾은 본문을 이번 프롬프트에 넣기

수업 §3의 검색+원본 유지, §4의 답변+근거 반환을 그래프 상태로 표현했습니다.

FLOW / 흐름을 먼저 읽으세요



Retrieval

리뷰 내용으로 관련 규정 후보를 검색합니다. 검색된 Document의 본문을 합쳐 policy_context를 만듭니다.

Augmentation

리뷰 JSON, 실제 규정 본문, 이전 초안, 검토 피드백과 수정 지시가 작성 프롬프트에 들어갑니다. 모델에게 문서를 읽었다고 말만 하는 것과 다릅니다.

Generation

draft가 답글을 쓰고 review도 같은 규정 본문을 읽습니다. 규정의 확인 절차를 환불 승인으로 바꾸지 않도록 지시합니다.

우리 코드의 차이

수업은 RunnablePassthrough.assign 등으로 원본과 문서를 보존합니다. 우리는 ReplyState의 review_text·policy_context·sources 필드로 같은 역할을 수행합니다.

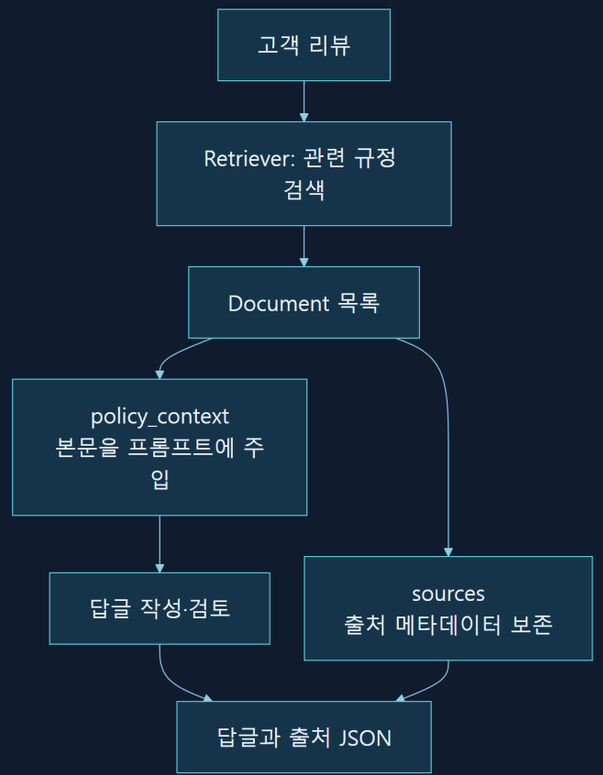
우리 코드 app/workflows/reply_graph.py:86 / retrieve

수업 원본 complete/07_rag_chain.ipynb · fa65250 (클릭)

07의 근거 반환이 sources가 된 과정

답글만으로는 어떤 규정이 들어갔는지 확인하기 어렵습니다.

FLOW / 흐름을 먼저 읽으세요



검색 결과에서 두 갈래

본문은 모델용 policy_context로 합치고, 출처는 사람과 API 클라이언트가 볼 sources로 남깁니다. 추가 검색을 한 번 더 실행하지 않습니다.

HTTP 응답의 출처

파일명, 절 제목, chunkId, policyVersion을 반환합니다. 로컬 파일 경로나 키 같은 서버 설정은 노출하지 않습니다.

출처 표시의 정확한 의미

sources는 모델에 제공한 검색 후보 목록입니다. 문장별 인용 정합성이나 의미적 참을 모두 검증했다는 뜻은 아닙니다. 향후 claim별 인용 검증과는 별도 기능입니다.

설명용 발췌·축약 코드 / 실제 함수는 페이지 하단 참조

```
context = join_document_texts(documents)
return {
    "policy_context": context,
    "sources": source_metadata,
}
# GraphReplyService.response -> ReplyResponse
```

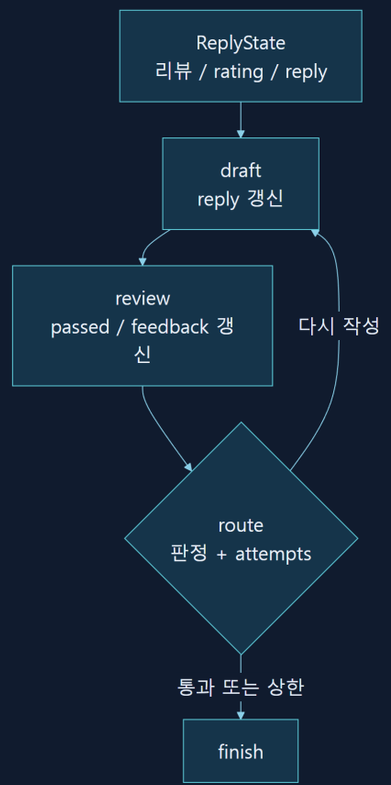
우리 코드 app/services/graph_reply_service.py:35 / response

수업 원본 complete/07_rag_chain.ipynb · fa65250 (클릭)

10 LangGraph: 상태·노드·엣지

수업 주문 계산 그래프의 구성 요소를 답글 업무로 바꿨습니다.

FLOW / 흐름을 먼저 읽으세요



상태: 작업 노트

ReplyState는 원본 리뷰·별점·초안·피드백·횟수를 공유합니다. TypedDict는 개발자와 도구에 필드 형태를 설명하고 모델 출력 검증은 Pydantic이 담당합니다.

노드: 한 단계의 작업

draft는 본문과 횟수를, review는 판정과 피드백을, retrieve는 규정 본문과 출처를 갱신합니다. 각 노드는 필요한 변경분만 반환합니다.

엣지: 다음 단계

add_edge는 고정 순서, add_conditional_edges는 판정에 따른 분기입니다. 모델이 임의로 그래프 밖의 도구를 실행하는 구조가 아닙니다.

덮어쓰기와 누적

이번 답글 상태의 reply·feedback은 최신 값으로 덮어씁니다. 수업의 리스트 누적 리듀서를 모든 필드에 붙이지 않았습니다. 체크포인터 이력과 필드 누적도 다릅니다.

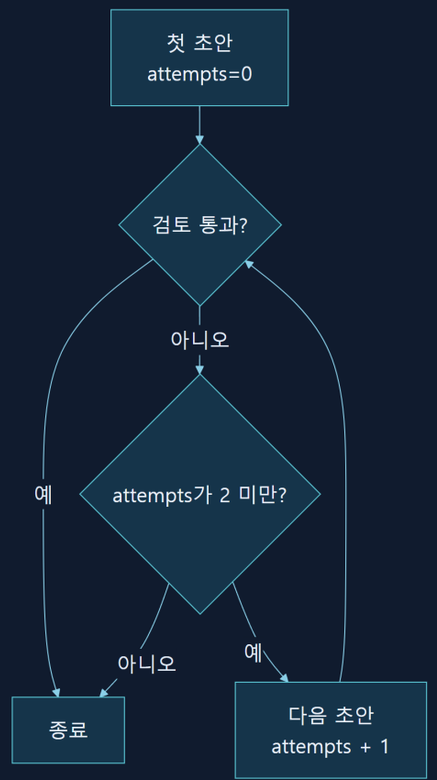
우리 코드 app/workflows/reply_graph.py:32 / ReplyState

수업 원본 complete/10_langgraph_basics.ipynb · fa65250 (클릭)

10의 반복 상한이 attempts가 된 과정

처음 한 번과 다시 쓰는 횟수를 구분합니다.

FLOW / 흐름을 먼저 읽으세요



횟수의 정의

처음 초안은 attempts=0입니다. 첫 재작성은 1, 두 번째 재작성은 2입니다. 따라서 생성은 최대 3회, 검토도 최대 3회 실행됩니다.

종료 기준

passed=true 또는 attempts>=2이면 finish로 갑니다. 검토가 계속 미통과여도 무한 루프가 되지 않습니다. recursion_limit은 별도 보조 방어입니다.

비용 범위

최대 생성 3회와 검토 3회가 가능하며 임베딩 비용은 별도입니다. 모델 max_retries=0으로 숨은 SDK 재시도를 막지만 토큰 상한만으로 전체 원화 비용이 고정되지는 않습니다.

설명용 발췌·축약 코드 / 실제 함수는 페이지 하단 참조

```
def route(state):
    if state["passed"] or state["attempts"] >= 2:
        return "finish"
    return "draft"
```

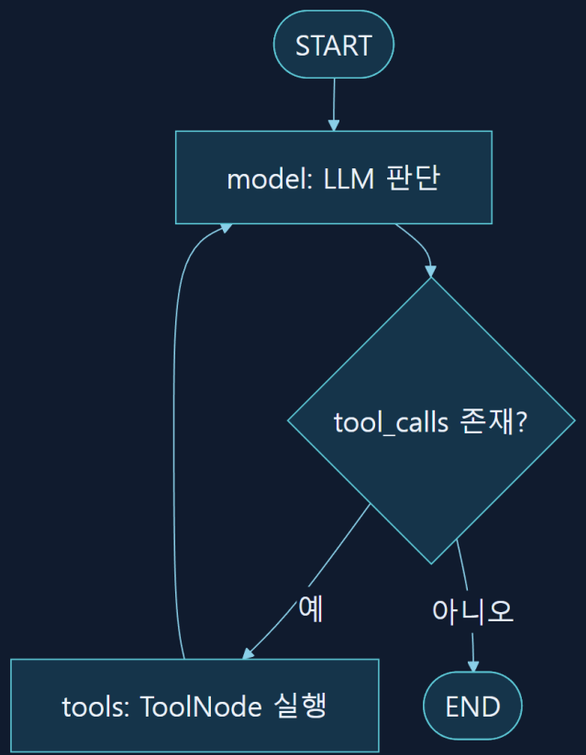
우리 코드 app/workflows/reply_graph.py:126 / route

수업 원본 complete/10_langgraph_basics.ipynb · fa65250 (클릭)

11 에이전트 수업에서 가져온 것과 남긴 것

수업의 model↔tools 루프와 우리 draft↔review 워크플로를 구별합니다.

FLOW / 흐름을 먼저 읽으세요



수업 그림의 의미

LLM 메시지에 tool_calls가 있으면 ToolNode가 도구를 실행하고 결과를 다시 모델에 줍니다. 도구 요청이 없으면 답변이 끝납니다.

우리 구현은 고정 워크플로

우리 그래프에는 ToolNode·bind_tools·자율 도구 선택이 없습니다. 규정 검색은 개발자가 정한 retrieve 노드에서 실행합니다. 그래프를 쓴다는 사실만으로 에이전트가 되는 것은 아닙니다.

실제로 가져온 핵심

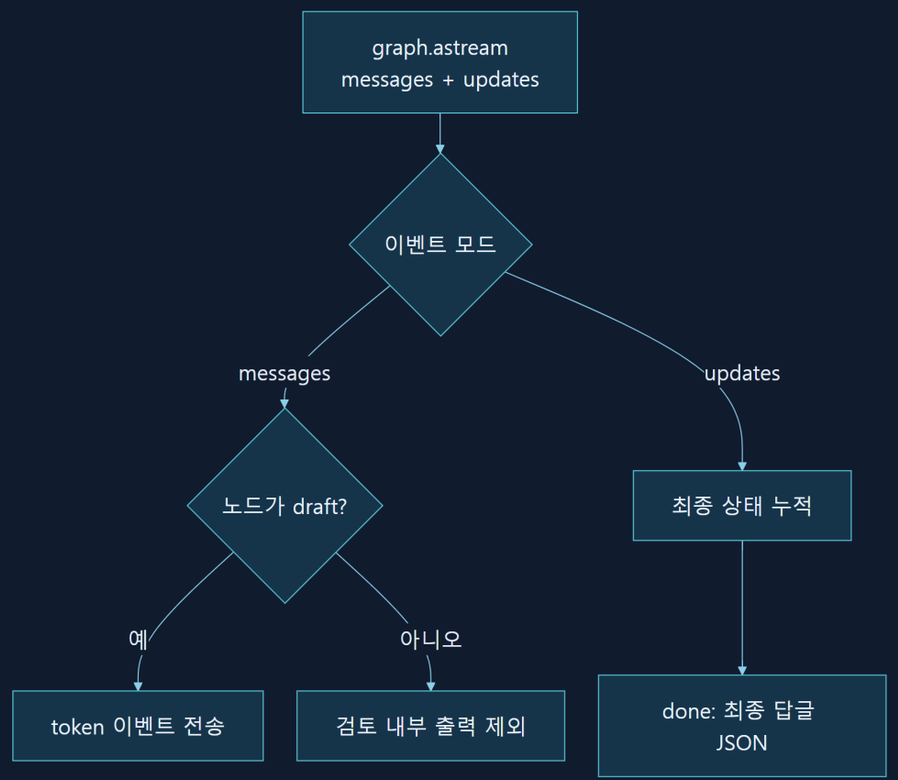
수업 §5·6의 messages 토큰, metadata의 노드 필터링, updates와의 다중 스트림, astream의 비동기 사용을 SSE에 적용했습니다.

우리 코드 app/services/graph_reply_service.py:48 / stream_reply
수업 원본 complete/11_langgraph_agent.ipynb · fa65250 (클릭)

11의 토큰 스트림이 SSE가 된 과정

모델이 생성하는 즉시 보내며 검토 모델 내부 출력은 제외합니다.

FLOW / 흐름을 먼저 읽으세요



messages 경로

(chunk, metadata)를 받아 langgraph_node가 draft일 때만 텍스트를 보냅니다. 수업의 model 필터를 우리 노드 이름인 draft로 바꿨습니다.

updates 경로

노드가 끝난 뒤의 상태 변경을 누적해 done 이벤트에 최종 JSON을 보냅니다. 그래프를 다시 호출해 최종 답글을 생성하지 않습니다.

초안과 최종본

재작성마다 draft_start가 오면 이전 임시 본문을 교체하세요. token은 검토 전 초안이며 done.reply가 최종본입니다. 검토 전 토큰에 부적절한 문장이 보일 가능성도 있어 자동 게시하면 안 됩니다.

설명용 발췌·축약 코드 / 실제 함수는 페이지 하단 참조

```
async for mode, data in graph.astream(
    inputs, stream_mode=["messages", "updates"]
):
    if mode == "messages":
        chunk, metadata = data
        if metadata.get("langgraph_node") == "draft":
            yield token_event(chunk)
```

우리 코드 app/services/graph_reply_service.py:48 / stream_reply

수업 원본 complete/11_langgraph_agent.ipynb · fa65250 (클릭)

SSE 오류는 상태 코드 대신 이벤트로 전달

헤더를 보낸 뒤에는 200을 502로 바꿀 수 없습니다.

FLOW / 흐름을 먼저 읽으세요



헤더 전 오류

라우터가 먼저 리뷰 존재를 확인합니다. 없는 ID는 기존 전역 처리기를 통해 404 JSON으로 반환됩니다. 일반 단건 API의 외부 호출 실패는 502입니다.

헤더 후 오류

스트리밍 도중 실패하면 error 이벤트에 AI_REQUEST_FAILED와 안전한 설명을 보내고 종료합니다. 이 경우 done을 보내지 않습니다.

취소와 확인 방법

CancelledError는 다시 전파해 실행 취소를 전달합니다. curl.exe -N 또는 실제 네트워크로 도착 시각을 확인합니다. TestClient의 모인 응답만으로 실시간성을 입증하지 않습니다.

설명용 발췌·축약 코드 / 실제 함수는 페이지 하단 참조

```
event: error
data: {"code":"AI_REQUEST_FAILED"}

# 정상 종료만 event: done
```

우리 코드 app/routers/review_router.py:34 / stream_reply

수업 원본 complete/11_langgraph_agent.ipynb · fa65250 (클릭)

12 자기 교정 RAG: 수업의 두 루프

검색을 고치는 일과 답변을 고치는 일은 다릅니다.

FLOW / 흐름을 먼저 읽으세요



첫 루프: 검색어 재작성

관련 문서가 없으면 질문의 검색 표현을 바꾸고 다시 검색합니다. rewrite_count가 이 루프를 제한합니다.

둘째 루프: 답변 재생성

문서가 있어도 답변의 숫자·조건이 틀릴 수 있습니다. 근거성 check가 미통과면 같은 근거로 다시 쓰며 generation_count가 제한합니다.

우리 단계와의 관계

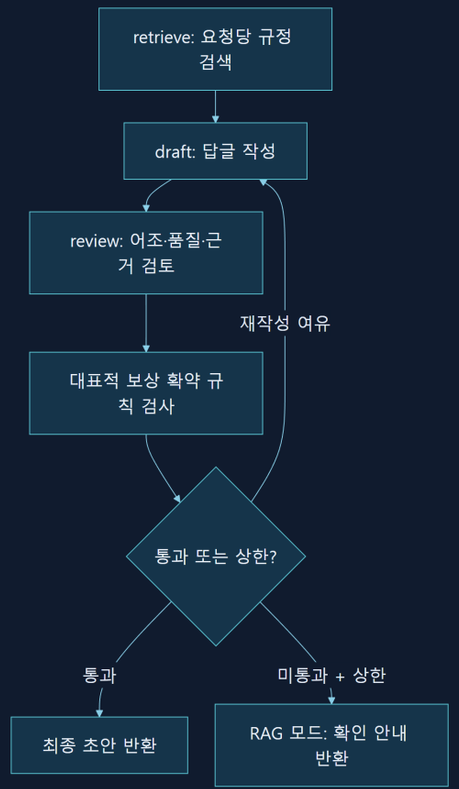
이번 과제는 답글 작성·자체 검토 루프가 우선입니다. 수업 전체 검색 교정 구조를 모두 구현했다고 표시하지 않습니다. 다음 페이지에서 채택 범위를 구분합니다.

우리 코드 app/workflows/reply_graph.py:58 / build_graph
수업 원본 complete/12_rag_graph.ipynb · fa65250 (클릭)

12가 우리 구현에 반영된 정확한 범위

구조화 검토 + 상한 + 확인 안내를 채택했습니다.

FLOW / 흐름을 먼저 읽으세요



반영한 기능

review가 reply와 실제 policy_context를 함께 읽어 품질·근거를 검토합니다. 실패 사유는 다음 draft에 전달하고 Python 카운터가 최대 재작성을 제한합니다.

추가한 방어

대표적인 쿠폰·전액 환불 확약은 규칙으로도 검사합니다. RAG 모드에서 상한까지 미통과면 상세 확약을 담지 않은 확인 안내로 바꿉니다. passed=false는 유지합니다.

아직 넣지 않은 기능

문서별 RelevanceGrade, 관련 문서만 남기는 필터, 검색어 rewrite 재검색, 별도 두 예산은 구현하지 않았습니다. top-k 결과가 무관할 가능성은 남습니다.

검증 한계

정규식은 바꿔쓰기를 놓치거나 부정 문장을 과차단할 수 있습니다. LLM 검토도 완벽하지 않습니다. 실제 규정 골든셋으로 품질을 측정해야 합니다.

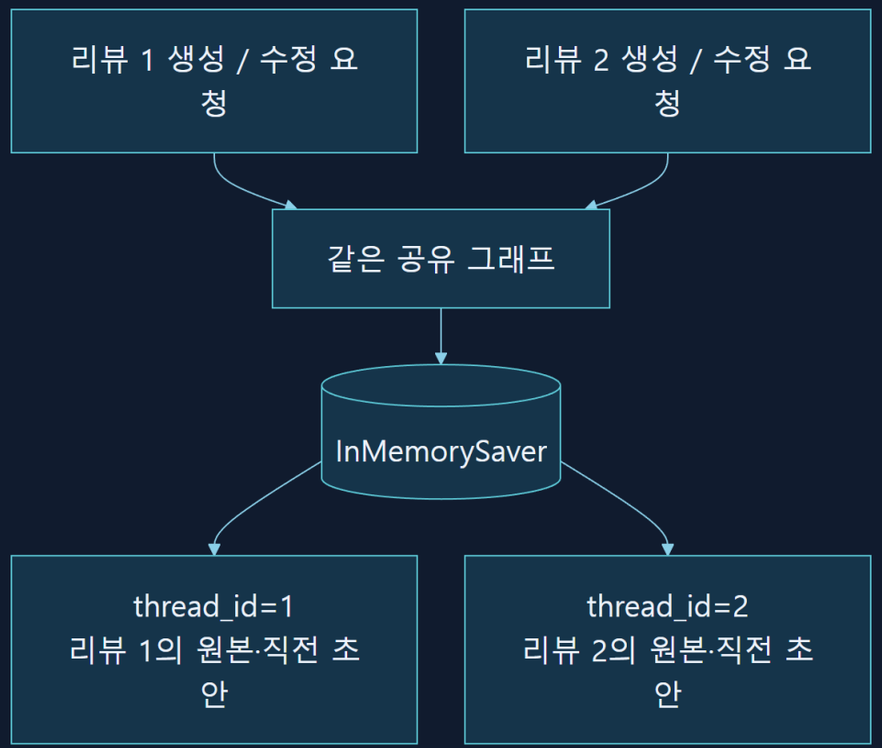
우리 코드 app/workflows/reply_graph.py:111 / review

수업 원본 complete/12_rag_graph.ipynb · fa65250 (클릭)

13 메모리: 리뷰 ID를 기억의 서랍 키로

thread_id는 OS 스레드나 MainThread와 다른 개념입니다.

FLOW / 흐름을 먼저 읽으세요



체크포인터

lifespan에서 만든 InMemorySaver를 그래프 compile에 연결합니다. 노드 실행 후 상태를 보관하고 다음 호출에서 복원합니다.

thread_id

리뷰 ID를 문자열로 사용합니다. 1번 리뷰의 초안과 2번 리뷰의 초안은 다른 상태로 저장됩니다. 같은 공유 그래프를 사용해도 서랍이 다릅니다.

실행 스레드와 구별

MainThread는 실제 요청 코드가 실행되는 스레드 이름입니다. thread_id는 어떤 기억을 읽을지 결정하는 논리 키입니다. 이름이 비슷하지만 서로를 대체하지 않습니다.

설명용 발췌·축약 코드 / 실제 함수는 페이지 하단 참조

```
saver = InMemorySaver()
graph = builder.compile(checkpointer=saver)
config = {"configurable": {"thread_id": str(review_id)}}
```

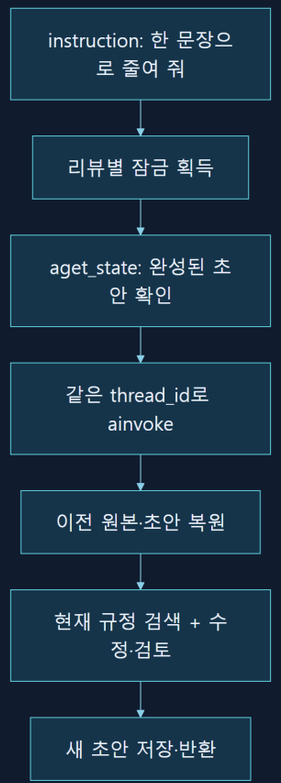
우리 코드 app/services/memory_reply_service.py:14 / config

수업 원본 complete/13_memory.ipynb · fa65250 (클릭)

13의 후속 질문이 우리 수정 API가 된 과정

클라이언트는 원본 리뷰 대신 instruction 한 줄만 보냅니다.

FLOW / 흐름을 먼저 읽으세요



수정 요청의 데이터

ReplyRevisionRequest는 instruction만 받습니다. 서버가 aget_state로 완료된 초안이 있는지 확인하고 같은 thread_id로 새 지시를 실행합니다.

이전 초안은 어디서 오나요?

체크포인터의 reply·review_text·rating이 복원됩니다. draft 템플릿은 previous_reply에 직전 답글을 넣습니다. 수정 요청마다 draft_count와 attempts를 새로 시작합니다.

HTTP 규칙

없는 리뷰는 404, 먼저 만든 초안이 없으면 409, 빈 지시는 422입니다. 같은 리뷰의 겹친 요청은 asyncio.Lock으로 순서를 지킵니다.

설명용 발췌·축약 코드 / 실제 함수는 페이지 하단 참조

```
POST /api/reviews/1/reply/revise
{"instruction":"한 문장으로 줄여 줘"}

await graph.ainvoke(
    {"revision_instruction": instruction,
     "draft_count": 0, "completed": False}, config
)
```

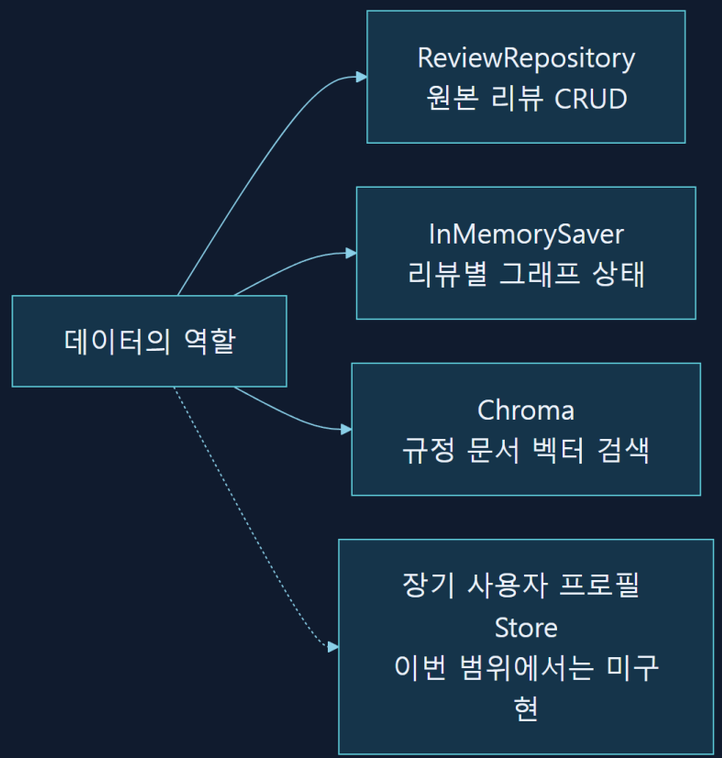
우리 코드 app/services/memory_reply_service.py:31 / revise_reply

수업 원본 complete/13_memory.ipynb · fa65250 (클릭)

13의 Store와 우리 세 보관함을 구별하기

이름이 비슷해도 저장 목적과 수명은 다릅니다.

FLOW / 흐름을 먼저 읽으세요



ReviewRepository

원본 리뷰를 등록·조회하는 업무 저장소입니다. 현재는 메모리이므로 재시작하면 초기화됩니다.

InMemorySaver

리뷰별 그래프 상태를 기억합니다. 프로세스 재시작으로 사라집니다. 최신 필드가 짧아도 이전 체크포인트 이력은 계속 쌓일 수 있습니다.

Chroma

규정 본문과 벡터를 검색합니다. 실제 모드의 디스크 인덱스가 남아 있어도 이전 리뷰 초안 기억이 자동 복원되는 것은 아닙니다.

수업의 확장 중 미구현

장기 사용자 프로필 Store, trim_messages, SummarizationMiddleware는 이번 범위에 없습니다. InMemoryStore도 이름의 장기와 별개로 프로세스 재시작 영속성을 보장하지 않습니다.

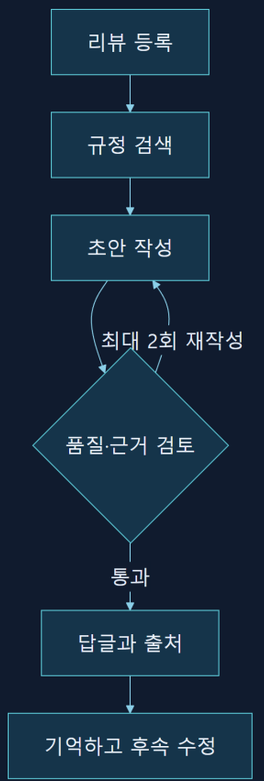
우리 코드 app/main.py:18 / create_app

수업 원본 complete/13_memory.ipynb · fa65250 (클릭)

환불 리뷰 한 건을 끝까지 따라가기

요구와 승인, 초안과 최종본, 검색 후보와 검증된 사실을 구별합니다.

FLOW / 흐름을 먼저 읽으세요



1. 입력과 검색

별점 1점, 배달 지연·환불 요구를 저장합니다. review_text를 임베딩해 환불 요청 검토 절차 등 후보 네 조각을 찾습니다.

2. 작성과 검토

리뷰와 규정을 읽어 사과와 담당자 확인 절차를 안내합니다. 환불을 요구했다는 사실이 환불 승인 근거가 되지는 않습니다. 검토가 미통과면 정해진 횟수 안에서 다시 씁니다.

3. 반환과 기억

reply·tone·needsFollowup·passed·attempts·sources를 반환하고 thread_id=1에 상태를 저장합니다. 한 문장으로 줄이라는 후속 지시만으로 직전 초안을 다듬습니다.

설명용 발췌·축약 코드 / 실제 함수는 페이지 하단 참조

```
{
  "tone": "사과",
  "needsFollowup": true,
  "passed": true,
  "attempts": 0,
  "sources": [{"chunkId":"policy-..."}]
}
```

우리 코드 tests/test_policy_rag.py:54 / test_rag_api_sources_revision_and_stream

수업 원본 complete/07_rag_chain.ipynb · fa65250 (클릭)

전부 async def로 바꾸는 것이 목표인가요?

목표는 요청 이벤트 루프를 오래 막는 작업을 없애는 것입니다.

FLOW / 흐름을 먼저 읽으세요



네이티브 비동기 우선

모델·원격 임베딩·그래프·SSE에는 ainvoke·aembed_query·astream을 사용합니다. async def 안에서 동기 HTTP 함수를 부르면 여전히 막힙니다.

동기 작업은 성격에 맞게 분리

로컬 Chroma·문서 읽기·분할은 작업 스레드에 넘깁니다. 큰 CPU 작업은 운영에서 프로세스나 작업 큐도 검토해야 하며 스레드가 처리량 증가를 보장하지는 않습니다.

짧은 계산은 동기로

조건 분기·Pydantic 검증·짧은 JSON 변환은 동기로 둡니다. 기존 등록·조회 def 라우터는 FastAPI가 작업 스레드에서 실행합니다.

판단 기준

I/O를 기다리는가? 라이브러리가 실제 async를 지원하는가? 동시 요청을 막는가? MainThread 로그는 참고 증거이며 무블로킹의 단독 증명은 아닙니다.

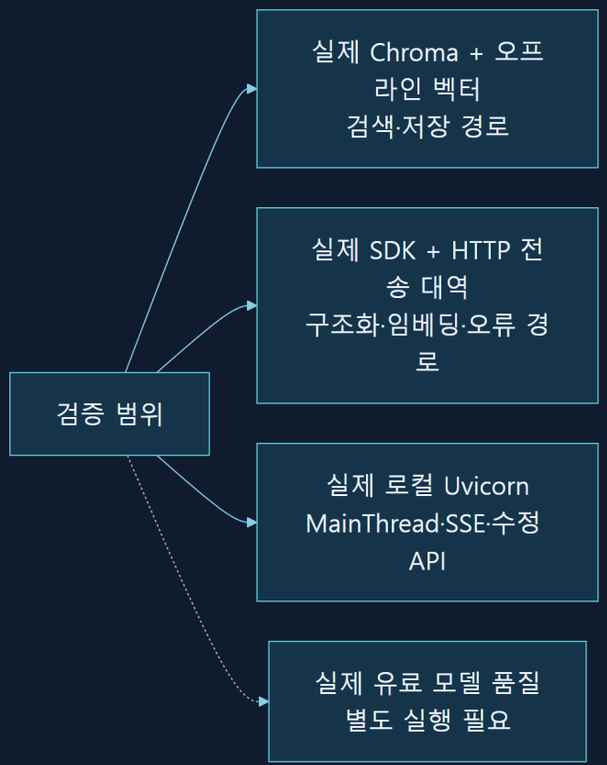
우리 코드 docs/ASYNC_DECISIONS.md

수업 원본 complete/11_langgraph_agent.ipynb · fa65250 (클릭)

검증한 사실과 별도로 확인할 품질

fake 결과를 실제 모델의 의미 정확도나 실제 비용으로 표시하지 않았습니다.

FLOW / 흐름을 먼저 읽으세요



자동 테스트 38개 + 하위 7개

기존 API·502 변환·생명 주기·구조화 출력·재작성 상한·SSE 오류·기억 격리·실제 Chroma 검색·영구 인덱스 재사용을 확인했습니다.

실제 로컬 서버

Uvicorn의 답글·초안·검토·검색·수정 MainThread, 한 번의 모델 생성, 시간차 토큰 전달, 출처와 한 문장 수정 응답을 확인했습니다.

검증하지 않은 것

유료 모델 의미 정확도, 실제 임베딩 검색 성능, 모든 환각·프롬프트 공격 차단, 운영 다중 worker·권한·보존기간은 별도입니다. 하위 라이브러리의 deprecation 경고 14건은 기능 실패와 구분합니다.

설명용 발췌·축약 코드 / 실제 함수는 페이지 하단 참조

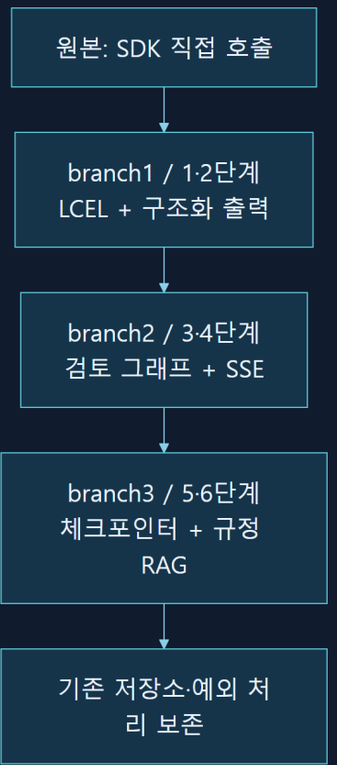
```
uv sync --python 3.14
uv run pytest -q
uv run python scripts/verify_local.py
uv run uvicorn app.main:app --port 8011
```

우리 코드 tests/test_policy_persistence.py
수업 원본 complete/05_embedding.ipynb · fa65250 (클릭)

코드에서 주석을 찾아 복습하기

짧은 주석은 입구이고, 상세 설명은 단계별 Markdown으로 이어집니다.

FLOW / 흐름을 먼저 읽으세요



검색과 임베딩

app/rag/policy_index.py의 PolicyRetriever·split_policy·build_policy_index 주석에서 수업 05·06·07의 대응 지점을 찾습니다.

그래프와 스트림

workflows/reply_graph.py의 ReplyState·review·route·compile 주석은 10·12·13으로 연결됩니다. graph_reply_service.py의 다중 스트림 주석은 11로 연결됩니다.

기억과 상세 문서

memory_reply_service.py의 config·aget_state 주석은 13으로 연결됩니다. docs/01~06과 LECTURE_TO_CODE.md에서 전체 흐름을 다시 볼 수 있습니다.

그림의 출처

수업 저장소 images/*.md의 Mermaid와 노트북 내용을 바탕으로 이 프로젝트의 노드·비동기 실행·HTTP 경계에 맞게 재구성했습니다. 다음 페이지에 정확한 원본 링크를 적었습니다.

우리 코드 docs/LECTURE_TO_CODE.md

수업 원본 images/12_1_0_rag_graph_flow.md · fa65250 (클릭)

참고한 수업 원본

2026-11-13 조회한 원격 최신 커밋 fa6525000cdfe2428b75e14236c2880dbc54b195

05 · 임베딩

complete/05_embedding.ipynb — §2 문장 벡터, §4 검색, §7 골든셋 비교. images/05_4_1_in_memory_semantic_search.md.

06 · 벡터 스토어

complete/06_vectorstore_and_retriever.ipynb — §3 저장, §4 검색, §6 retriever, §7 다시 열기. images/06_6_1_vectorstore_vs_retriever.md.

07 · RAG 체인

complete/07_rag_chain.ipynb — §3 LCEL, §4 근거 반환, §5 프롬프트 한계. images/07_3_1_rag_chain_parallel.md, 07_4_1_rag_chain_with_basis.md.

10 · 그래프 기초

complete/10_langgraph_basics.ipynb — §2 상태·노드·엣지, §4 리듀서, §5 반복 상한. images/10_5_0_kitchen_loop.md.

11 · 에이전트와 스트림

complete/11_langgraph_agent.ipynb — §1 model/tools, §5 다중 스트림, §6 astream. images/11_1_0_agent_graph_structure.md.

12 · 자기 교정 RAG

complete/12_rag_graph.ipynb — §3 채점기, §4 노드, §5 두 루프, §7 독립 검증. images/12_1_0_rag_graph_flow.md.

13 · 메모리

complete/13_memory.ipynb — §1 thread 격리, §2 상태 조회, §3 컨텍스트 관리, §4 Store. images/13_1_3_thread_isolation.md, 13_4_1_checkpointer_vs_store.md.